

CoughSense Codebase Analysis

Analysis Date: December 6, 2025
Project: CoughSense - Dry vs Wet Cough Classifier
Location: /home/ubuntu/coughsense_project/Coughsense-main/

Executive Summary

CoughSense is a web-based application for classifying cough sounds as either “dry” or “wet”. The application currently uses **mock predictions** that need to be replaced with a real machine learning model. This document provides a comprehensive analysis of the codebase structure, technology stack, and integration points for implementing the ML model.

1. Directory Structure

```
Coughsense-main/
├── public/                                # Static assets
│   ├── favicon.ico
│   ├── placeholder.svg
│   └── robots.txt
├── src/                                  # Application source code
│   ├── components/                      # React components
│   │   ├── ui/                         # Shadcn UI components (40+ components)
│   │   ├── NavLink.tsx                 # Navigation link component
│   │   └── WaveformVisualizer.tsx      # Audio visualization component
│   ├── hooks/                          # React custom hooks
│   │   ├── use-mobile.tsx
│   │   └── use-toast.ts
│   ├── integrations/                   # Third-party integrations
│   │   └── supabase/
│   │       ├── client.ts               # Supabase client configuration
│   │       └── types.ts                 # TypeScript types for Supabase
│   ├── lib/                            # Utility functions
│   │   └── utils.ts
│   ├── pages/                          # Application pages
│   │   ├── Index.tsx                  # Main page (recording/upload)
│   │   ├── Results.tsx                # Results display page
│   │   └── NotFound.tsx               # 404 page
│   ├── App.tsx                         # Main app component with routing
│   ├── main.tsx                       # Application entry point
│   ├── index.css                      # Global styles
│   └── supabase/                      # Supabase backend
│       ├── functions/
│       │   ├── predict-cough/
│       │   │   └── index.ts           # ⚠️ MOCK PREDICTION LOGIC HERE
│       └── config.toml
├── .env                                # Environment variables
├── package.json                       # Dependencies and scripts
├── vite.config.ts                     # Vite build configuration
└── README.md                          # (Empty)
```

2. Technology Stack

Frontend

- **Framework:** React 18.3.1 with TypeScript
- **Build Tool:** Vite 5.4.19
- **UI Library:** Radix UI components + Shadcn UI
- **Styling:** Tailwind CSS with custom theme
- **Routing:** React Router DOM 6.30.1
- **State Management:** React Hooks + TanStack Query 5.83.0
- **Form Handling:** React Hook Form 7.61.1 + Zod validation

Backend

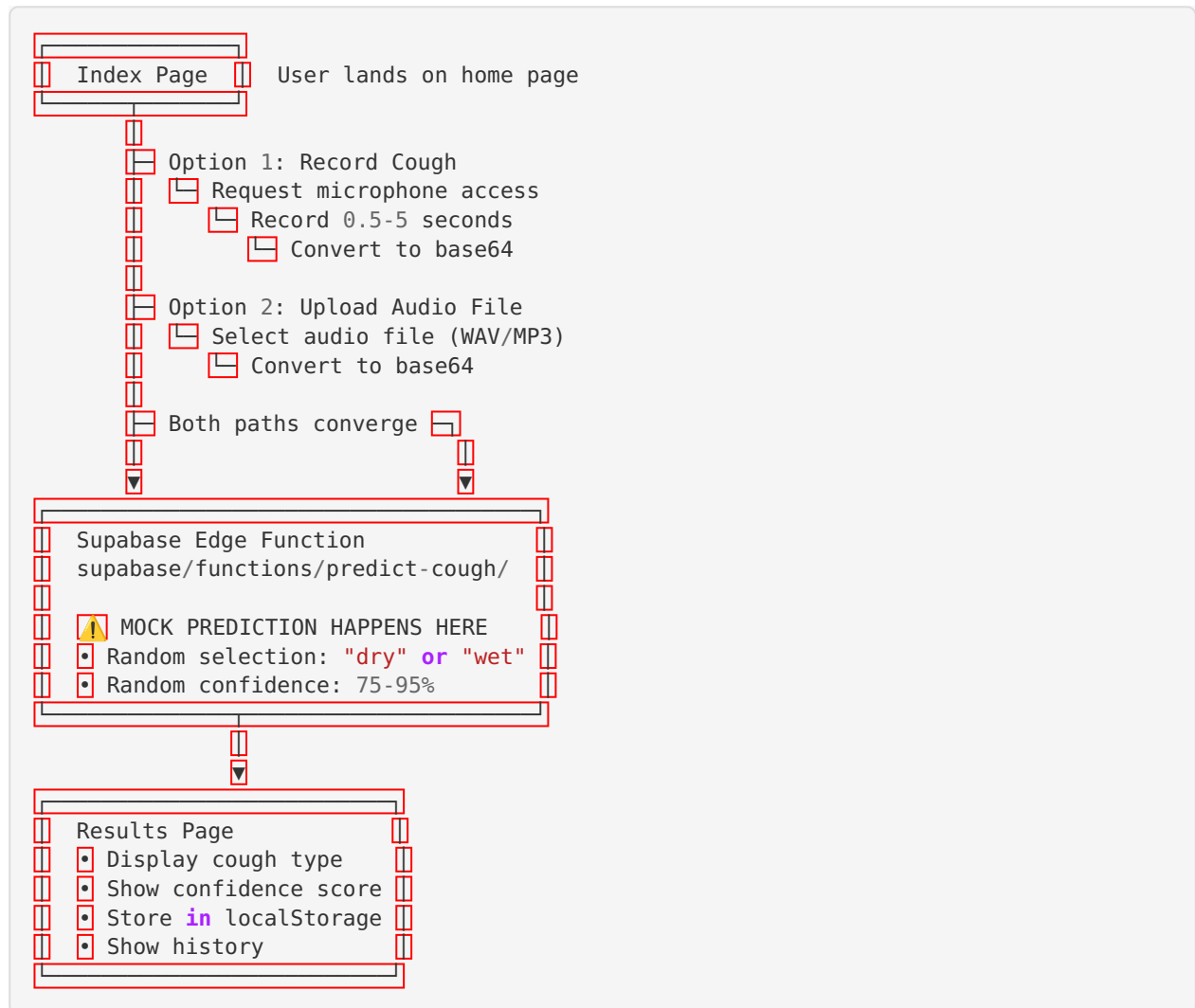
- **Platform:** Supabase (BaaS - Backend as a Service)
- **Edge Functions:** Deno runtime (TypeScript)
- **API:** Supabase Functions (serverless)

Audio Processing (Frontend)

- **Recording:** MediaRecorder API
 - **Format:** WebM with Opus codec
 - **Visualization:** Canvas API for waveform display
 - **Audio Context:** Web Audio API for level analysis
-

3. Application Flow

User Journey



4. Mock Prediction Implementation

Location

File: /supabase/functions/predict-cough/index.ts

Lines: 64-68

Current Implementation

```

// MOCK PREDICTION (Replace with actual model)
// Only 2 classes: dry or wet
const coughTypes = ['dry', 'wet'];
const randomType = coughTypes[Math.floor(Math.random() * coughTypes.length)];
const confidence = 75 + Math.random() * 20; // 75-95%

```

How It Works

1. Receives base64-encoded audio data via HTTP POST

2. **Currently:** Randomly selects “dry” or “wet”
3. **Currently:** Generates random confidence between 75-95%
4. Adds 1-second artificial delay to simulate processing
5. Returns JSON response with prediction

Response Format

```
{
  predicted_cough_type: "dry" | "wet",
  confidence_score: number, // 75.0 to 95.0
  message: "Mock prediction - replace with actual ML model"
}
```

5. Audio File Handling

Recording Process (Index.tsx)

1. User Initiates Recording

- **Function:** `startRecording()` (lines 52-99)
- **Microphone Settings:**

```
typescript
{
  audio: {
    echoCancellation: true,
    noiseSuppression: true,
    sampleRate: 44100
  }
}
```

2. Recording Configuration

- **Format:** `audio/webm; codecs=opus`
- **Duration:** 0.5 to 5 seconds (enforced by timer)
- **Real-time Monitoring:** FFT analysis for waveform visualization

3. Audio Conversion

- **Function:** `processAudio()` (lines 109-151)
- **Steps:**
 1. Validate minimum duration (0.5 seconds)
 2. Convert Blob to base64 using `FileReader`
 3. Extract base64 data (remove data URL prefix)
 4. Send to Supabase function

4. File Upload Alternative

- **Function:** `handleFileUpload()` (lines 153-164)
- **Accepts:** Any audio/* MIME type
- **Same processing:** Uses `processAudio()` function

Audio Data Format Sent to Backend

```
{
  audio: "<base64_encoded_audio_data>"
}
```

Note: The audio data is sent as a base64 string WITHOUT the data URL prefix (i.e., without `data:audio/webm;base64,`)

6. Input/Output Format for ML Model

Input Format

Current Input to Function:

```
{
  "audio": "GkXfo59ChoEBQveBAULygQRC84EIQoKEd2VibUKHgQRChYECGF0AZwH/..."
}
```

Audio Characteristics:

- **Encoding:** Base64 string
- **Original Format:** WebM with Opus codec (from recording) OR original file format (from upload)
- **Sample Rate:** 44100 Hz (for recorded audio)
- **Duration:** 0.5 to 5 seconds typically

Expected Output Format

Current Response Structure:

```
interface PredictionResponse {
  predicted_cough_type: "dry" | "wet";
  confidence_score: number; // 0-100
  message?: string;        // Optional message
}
```

Example Response:

```
{
  "predicted_cough_type": "wet",
  "confidence_score": 87.3,
  "message": "Prediction completed successfully"
}
```

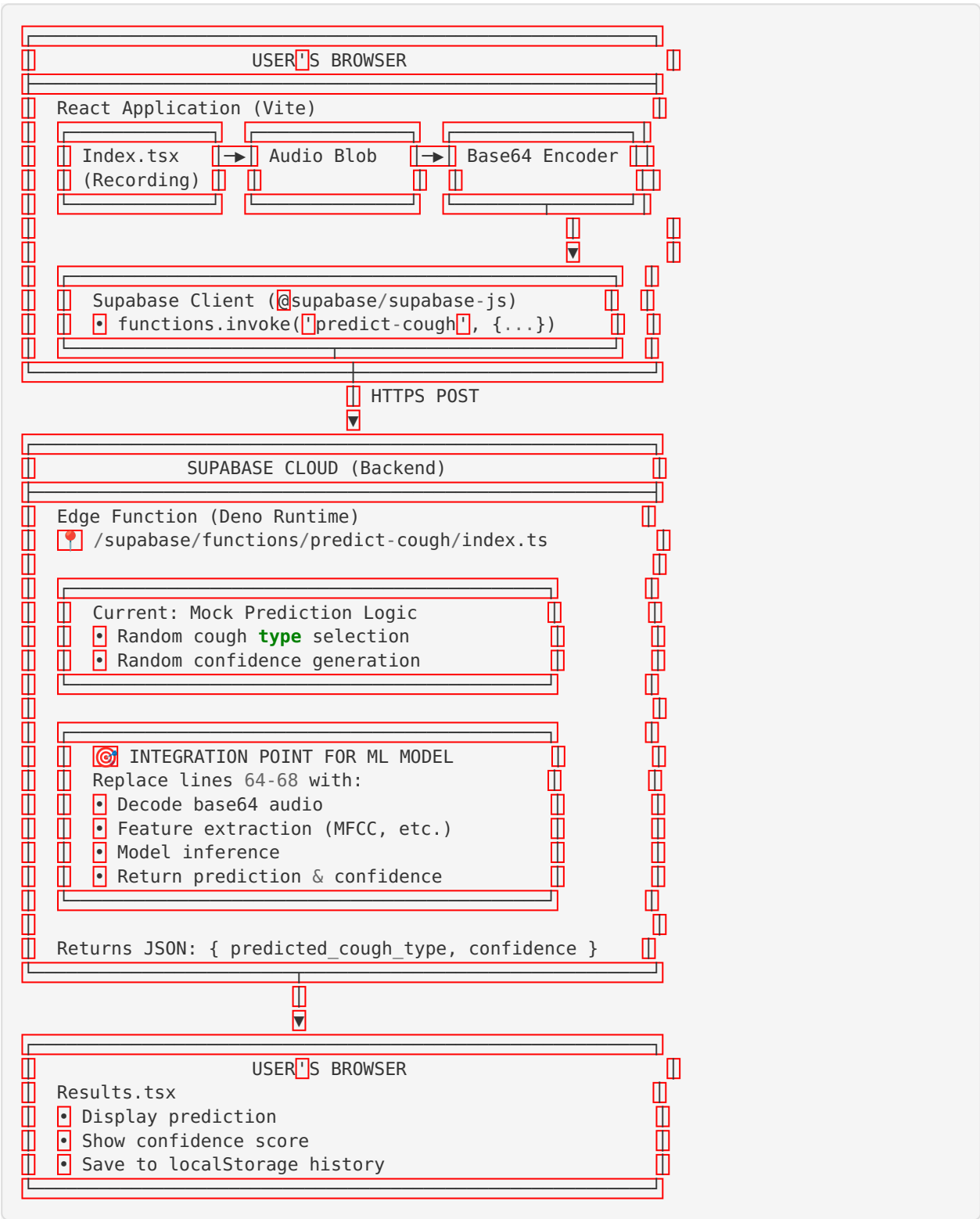
Frontend Expectation

The frontend (`Index.tsx` , lines 132-137) expects:

- `data.predicted_cough_type` - Must be exactly "dry" or "wet"
- `data.confidence_score` - Number between 0-100
- Any other fields are optional and ignored

7. Architecture Overview

System Architecture Diagram



Key Architectural Decisions

- Serverless Architecture:** Using Supabase Edge Functions eliminates need for server management
- Client-Side Audio Processing:** All recording/encoding happens in browser
- No Database Storage:** Results stored only in browser's localStorage

4. **Stateless Backend:** Each prediction is independent, no session management

8. What Needs to be Replaced/Modified

Critical Changes Required

1. Primary Target: Supabase Edge Function

File: /supabase/functions/predict-cough/index.ts

Lines to Replace: 64-68 (mock prediction logic)

Current Code:

```
const coughTypes = ['dry', 'wet'];
const randomType = coughTypes[Math.floor(Math.random() * coughTypes.length)];
const confidence = 75 + Math.random() * 20;
```

Needs to Become:

```
// Decode base64 audio
// Extract audio features (MFCC, spectral features, etc.)
// Load ML model
// Run inference
// Return actual prediction and confidence
```

2. Implementation Options

Option A: Python Microservice (Recommended)

- Create a separate Python service with scikit-learn/TensorFlow
- Edge function calls this service via HTTP
- **Pros:** Full access to Python ML libraries (librosa, scikit-learn)
- **Cons:** Need to deploy and manage Python service

Option B: ONNX Runtime in Deno

- Convert model to ONNX format
- Use onnxruntime-node in Deno
- **Pros:** Everything stays in Supabase Edge Function
- **Cons:** Feature extraction still needed, limited library support

Option C: TensorFlow.js

- Convert model to TensorFlow.js format
- Run inference directly in Edge Function
- **Pros:** No external dependencies
- **Cons:** Model must be TensorFlow-compatible

Integration Checklist

- [] Decode base64 audio to binary format
- [] Convert audio format if needed (WebM → WAV)
- [] Extract audio features:
- [] MFCC (Mel-frequency cepstral coefficients)

- [] Spectral centroid
- [] Spectral bandwidth
- [] Zero-crossing rate
- [] Other features used during training
- [] Load trained ML model
- [] Prepare feature vector in correct format
- [] Run model inference
- [] Map model output to “dry” or “wet”
- [] Calculate confidence score (0-100)
- [] Return response in expected format
- [] Handle errors gracefully
- [] Add logging for monitoring

9. Frontend Integration Points

No Changes Required in Frontend

The frontend is already designed to work with a real ML model. As long as the response format remains the same, **no frontend changes are necessary**.

Response Contract

```
// Edge function must return this structure
{
  predicted_cough_type: "dry" | "wet",
  confidence_score: number, // 0-100
  message?: string         // Optional
}
```

Error Handling

The frontend already handles errors from the backend:

```
if (error) throw error; // Caught and displayed to user
```

10. Environment Configuration

Supabase Configuration

Environment Variables (`.env`):

```
VITE_SUPABASE_PROJECT_ID="eqqzgxfikwcxofmahrrg"
VITE_SUPABASE_URL="https://eqqzgxfikwcxofmahrrg.supabase.co"
VITE_SUPABASE_PUBLISHABLE_KEY="eyJhbGc..."
```

Supabase Project: eqqzgxfikwcxofmahrrg

Deployment Configuration

Supabase CLI Usage:

```
# Deploy Edge Function
supabase functions deploy predict-cough

# Test locally
supabase functions serve predict-cough
```

11. Dependencies

Frontend Dependencies (Key Packages)

- `@supabase/supabase-js` : ^2.86.0 - Supabase client
- `react` : ^18.3.1 - UI framework
- `react-router-dom` : ^6.30.1 - Routing
- `@tanstack/react-query` : ^5.83.0 - Data fetching
- `lucide-react` : ^0.462.0 - Icons
- `tailwindcss` : ^3.4.17 - Styling

Backend Dependencies (Deno)

- Standard library: <https://deno.land/std@0.168.0>
- Currently no ML libraries

To Add for ML Integration

Depending on chosen approach:

- **Python microservice:** Flask/FastAPI, librosa, scikit-learn/TensorFlow, numpy
- **ONNX:** onnxruntime-node
- **TensorFlow.js:** @tensorflow/tfjs-node

12. Testing & Development

Current Scripts (package.json)

```
{
  "dev": "vite",           // Start development server
  "build": "vite build",    // Production build
  "preview": "vite preview", // Preview production build
  "lint": "eslint ."        // Run linter
}
```

Local Development Setup

```
# Install dependencies
npm install

# Start development server
npm run dev

# Start Supabase locally (if testing Edge Functions)
supabase start
supabase functions serve
```

Testing the Edge Function

```
# Example curl request
curl -X POST https://eqqzgxfikwcxofmahrrg.supabase.co/functions/v1/predict-cough \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <ANON_KEY>" \
  -d '{"audio": "<base64_audio_data>"}'
```

13. Data Flow Summary

Complete Data Flow

1. User Action
 - ☐ Click "Record" OR Upload file
2. Audio Capture
 - ☐ MediaRecorder API (WebM/Opus, 44.1kHz)
 - ☐ Duration: 0.5-5 seconds
3. Client-Side Processing
 - ☐ Blob ☐ FileReader ☐ Base64
 - ☐ Remove data URL prefix
 - ☐ Validation (minimum duration)
4. API Request
 - ☐ POST to Supabase Edge Function
 - ☐ Body: { audio: "<base64_string>" }
 - ☐ Headers: Content-Type, Authorization
5. Server-Side Processing (TO BE IMPLEMENTED)
 - ☐ Decode base64
 - ☐ Audio format conversion (if needed)
 - ☐ Feature extraction
 - ☐ ML model inference
 - ☐ Confidence calculation
6. Response
 - ☐ JSON: { predicted_cough_type, confidence_score }
 - ☐ HTTP 200 on success, 500 on error
7. Client-Side Rendering
 - ☐ Store in localStorage
 - ☐ Navigate to Results page
 - ☐ Display prediction with UI
 - ☐ Show in history list

14. Security Considerations

Current Security

- **Authentication:** Uses Supabase anon key (public)
- **CORS:** Allows all origins (*)
- **Data Storage:** Client-side only (localStorage)
- **No sensitive data:** Audio not permanently stored

Recommendations for ML Integration

- Consider rate limiting on Edge Function
- Add request size limits (prevent large audio uploads)
- Validate audio format server-side
- Add monitoring/logging for abuse detection
- Consider authentication if making production app

15. Next Steps for ML Integration

Recommended Implementation Plan

Phase 1: Model Preparation

1. Train/validate your ML model locally
2. Determine required audio features
3. Test model with sample audio files
4. Document expected input feature format
5. Export model in appropriate format

Phase 2: Backend Development

1. Choose integration approach (Python microservice recommended)
2. Set up audio decoding (base64 → binary)
3. Implement feature extraction pipeline
4. Load and test model inference
5. Wrap in API endpoint

Phase 3: Integration

1. Update Edge Function to call ML service
2. Handle response mapping
3. Implement error handling
4. Add logging/monitoring

Phase 4: Testing

1. Test with recorded audio samples
2. Test with uploaded files
3. Validate confidence scores
4. Load testing
5. Error scenario testing

Phase 5: Deployment

1. Deploy ML service (if using microservice)
2. Update Edge Function
3. Test in production
4. Monitor performance

16. Additional Notes

Strengths of Current Implementation

- ✓ Clean separation of concerns
- ✓ Responsive UI with good UX
- ✓ Real-time audio visualization
- ✓ Proper error handling
- ✓ Local history tracking

- ✓ Type-safe TypeScript codebase
- ✓ Modern React patterns (hooks, composition)

Limitations to Address

- ⚠ No backend audio storage
- ⚠ No user authentication
- ⚠ No analytics/telemetry
- ⚠ Limited audio format support documentation
- ⚠ No model versioning
- ⚠ No A/B testing capability

Questions for ML Integration

1. What audio features does your model expect?
2. What format is your trained model in?
3. What were the training data characteristics (sample rate, duration)?
4. What accuracy/performance is expected?
5. Should audio be stored for model improvement?

17. File Modification Priority

High Priority (Must Modify)

1. ★ `/supabase/functions/predict-cough/index.ts` - Replace mock logic

Medium Priority (May Modify)

1. `/supabase/config.toml` - Add function environment variables if needed
2. `/.env` - Add ML service URL if using microservice

Low Priority (Optional)

1. `/src/pages/Index.tsx` - Add more audio metadata if needed
2. `/src/pages/Results.tsx` - Enhanced result display

No Modification Needed

- All other frontend files ✓
- UI components ✓
- Routing ✓
- Build configuration ✓

Conclusion

The CoughSense application has a well-structured, modern architecture that makes ML model integration straightforward. The primary task is replacing the 5 lines of mock prediction code in the Supabase Edge Function with actual ML inference logic. The frontend is already production-ready and requires no changes.

Key Takeaway: Focus all ML integration efforts on `/supabase/functions/predict-cough/index.ts` lines 64-68, ensuring the response format matches the expected structure.

Document Version: 1.0

Last Updated: December 6, 2025

Prepared By: DeepAgent AI Analysis